

Alterações no Back-end

BuscaML API — projeto ASP.NET Core (.NET 10)

Documento gerado em 08/09/2026 · commits `1f60013` (Sprint 3) e `ded76b1` (integração front)

1. Objetivo da etapa

Construir a camada que conversa com o Mercado Livre e expõe os dados por uma API REST: buscar produtos, guardar em banco com cache, cruzar com o ranking de “mais vendidos” e devolver tudo em JSON para o front consumir. Além disso, deixar o projeto rodável **sem depender de credencial nem de internet** durante o desenvolvimento.

2. Arquivos criados

Camada de dados (EF Core + SQLite)

Arquivo	O que contém
<code>Models/Produto.cs</code>	Produto do ML. Chave primária = <code>IdML</code> (o código “MLB...”). Guarda preço, categoria, frete grátis, imagem, link, se é mais vendido e a posição no ranking.
<code>Models/Categoria.cs</code>	Categoria do ML. Chave primária = o próprio ID da categoria (ex.: <code>MLB1051</code>).
<code>Models/Busca.cs</code>	Registro de cada pesquisa feita (termo normalizado + data/hora). Base do cache.
<code>Models/ResultadoBusca.cs</code>	Liga uma <code>Busca</code> aos produtos que ela retornou, com o preço no momento.
<code>Models/HighlightCache.cs</code>	Guarda a resposta crua do endpoint de ranking (<code>/highlights</code>) por categoria.
<code>Data/AppDbContext.cs</code>	Mapeamento das 5 tabelas. Conversores especiais: <code>JsonDocument</code> gravado como texto; <code>DateTimeOffset</code> convertido para um formato que o SQLite consegue ordenar.
<code>Database/schema.sql</code>	Script SQL do banco, gerado a partir do modelo (referência).

Integração com o Mercado Livre

Arquivo	O que contém
<code>Services/IMercadoLivreClient.cs</code>	Interface: define <i>o que</i> é possível pedir ao ML (buscar, ranking, item, categoria) sem dizer <i>como</i> .
<code>Services/MercadoLivreClient.cs</code>	Implementação real: chama a API oficial <code>api.mercadolivre.com</code> via <code>HttpClient</code> .
<code>Services/FakeMercadoLivreClient.cs</code>	Implementação falsa: devolve 4 produtos fixos, sem rede. Usada em desenvolvimento/demonstração.
<code>Services/MercadoLivreAuthHandler.cs</code>	Adiciona o cabeçalho <code>Authorization: Bearer</code> em toda chamada. Aceita token fixo ou faz o fluxo OAuth <code>client_credentials</code> , renovando o token sozinho.

Regra de negócio e API

Arquivo	O que contém
Services/IBuscaService.cs / BuscaService.cs	O cérebro. Busca com cache por tempo (TTL); se falhar, devolve o último resultado salvo; cruza com o ranking para marcar <code>mais_vendido</code> ; persiste tudo no banco.
Services/HighlightsBackgroundService.cs	Serviço em segundo plano: de tempos em tempos atualiza o ranking de todas as categorias já vistas.
Controllers/BuscaController.cs	Endpoint <code>GET /api/buscar</code> .
Controllers/ProdutosController.cs	Endpoints <code>GET /api/produtos/{id}</code> e <code>GET /api/produtos/destaques</code> .
DTOs/MercadoLivreDtos.cs	Classes que espelham o JSON que o ML devolve.
DTOs/BuscaResponseDto.cs	Formato do JSON que a nossa API devolve (contrato com o front): <code>termo</code> , <code>total_resultados</code> , <code>resultados[]</code> .

3. Arquivos alterados

Arquivo	Mudança
Program.cs	Registro de tudo acima (injeção de dependência), configuração do banco, do <code>HttpClient</code> e do serviço em segundo plano. Escolha automática entre cliente real e fake. Passou a servir o front (<code>wwwroot</code>) e a liberar CORS. Seed inicial quando o banco está vazio.
appsettings.json	Seções novas: <code>MercadoLivre</code> (URL, site, <code>UseFake</code> , credenciais) e <code>Cache</code> (tempos de validade).
appsettings.Development.json	<code>MercadoLivre:UseFake = true</code> — em desenvolvimento usa o cliente fake.
BuscaML_API.csproj	Pacotes <code>Microsoft.EntityFrameworkCore.Sqlite</code> e <code>.Design</code> . <code>RootNamespace = BuscaML.Api</code> . <code>UserSecretsId</code> para guardar credenciais fora do repositório.

Arquivos removidos removido

`WeatherForecast.cs` e `Controllers/WeatherForecastController.cs` — exemplo que vem no template do .NET e não era usado.

4. Decisões técnicas

- **Banco SQLite** em arquivo (`buscaml.db`), criado automaticamente no startup (`EnsureCreated`). Não precisa instalar servidor de banco.
- **Produto identificado pelo IdML** (não por um número interno) — é o que o front e o ML usam.
- **Cache por TTL**: se o mesmo termo foi buscado há poucos minutos, responde do banco sem chamar o ML.

- **Fallback:** se o ML cair, a API devolve o último resultado salvo em vez de dar erro.
- **Cliente fake** ligado por configuração — o `BuscaService` não sabe se está falando com o ML real ou com dados fixos.
- **Autenticação isolada** num `DelegatingHandler` — o resto do código não se preocupa com token.

5. Endpoints da API

Método	Rota	Descrição
GET	<code>/api/buscar?termo=fone</code>	Busca produtos. Usa cache e fallback. <code>400</code> se faltar o termo; <code>404</code> se não houver dado nem cache.
GET	<code>/api/produtos/{id}</code>	Detalhe de um produto. Primeiro no cache local; se não tiver, busca no ML. <code>404</code> se não existir.
GET	<code>/api/produtos/destaques?quantidade=20</code>	Produtos marcados como mais vendidos, ordenados pela posição no ranking.

Formato da resposta (mesmo para busca e destaques):

```
{
  "termo": "fone",
  "total_resultados": 4,
  "resultados": [
    { "id": "...", "titulo": "...", "preco": 199.9, "categoria": "...",
      "mais_vendido": true, "frete_gratis": true,
      "imagem_url": "...", "link_ml": "..." }
  ]
}
```

6. Como testar

```
dotnet run
# depois, no navegador ou no BuscaML_API.http:
# http://localhost:5102/api/buscar?termo=fone
# http://localhost:5102/api/produtos/destaques
# http://localhost:5102/api/produtos/MLB-FAKE-2
```

Testado: build sem erros; 1ª busca popula o banco e cruza com o ranking; 2ª busca idêntica responde do cache; `destaques` vem ordenado; produto por ID vem do cache; ID inexistente retorna 404; sem credencial o sistema cai no fallback sem quebrar.